

Програмування мовою C++. Опрацювання одновимірних масивів. Пошук, сортування.

1. Умовні оператори C++

1.1. Оператор if

Використовується для перевірки умов та вибору однієї з гілок виконання.

Форма:

```
if (умова)  
    оператор;
```

```
if (умова)  
    оператор1;  
else  
    оператор2;
```

Вкладені if:

```
if (умова1)  
    ...  
else if (умова2)  
    ...  
else  
    ...
```

Типові помилки:

- пропущені дужки { }
- оператор = замість ==
- неправильний порядок перевірок

1.2. Оператор вибору switch

Зручний, коли є багато варіантів для однієї змінної.

Форма:

```
switch (вираз) {  
    case значення1: оператор; break;  
    case значення2: оператор; break;  
    ...  
    default: оператор;  
}
```

Особливості:

працює лише з цілочисельними + char

оператор break обов'язковий, щоб уникнути "провалювання" в наступні гілки

1.3. Оператори break і continue

break

завершує цикл або switch негайно.

continue

перериває поточну ітерацію циклу та переходить до наступної.

Приклад:

```
for (int i=0; i<10; i++) {  
    if (i == 5) continue;  
    if (i == 8) break;  
}
```

2. Цикли C++

2.1. Цикл for

Коли відома кількість повторень.

```
for (int i = 0; i < n; i++)  
    ...
```

2.2. Цикл while

Виконується, доки умова істинна.

while (умова)
оператор;

2.3. Цикл do...while

Гарантовано виконується хоча б один раз.

```
do {  
    ...  
} while (умова);
```

3. Масиви в C++

3.1. Що таке масив

Масив — це набір елементів **одного типу**, доступних за спільним ім'ям.

тип ім'я[розмір];

3.2. Одновимірні масиви

Опис:

```
int A[10];  
char M[20];  
double X[50];
```

Доступ до елементів:

```
A[0] = 15;  
A[9] = 22;
```

Ініціалізація:

```
int B[5] = {1, 2, 3, 4, 5};  
int C[] = {3, -1, 9};
```

Ініціалізація char-масиву (рядки):

```
char s[] = "Hello";  
char s2[] = { 'H', 'i', '\0' };
```

Обмеження:

C++ не контролює вихід за межі масиву → відповідальність програміста.

3.3. Двовимірні масиви (матриці)

Опис:

```
int M[3][4];  
float A[10][10];
```

Доступ:

```
M[0][3] = 8;  
M[2][1] = -9;
```

Ініціалізація:

```
int A[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

"Безрозмірна" ініціалізація:

```
int B[][4] = {  
    {1, 2},  
    {3},  
    {4, 5, 6, 7}  
};
```

3.4. Багатовимірні масиви

Приклад:

```
int T[3][5][6];  
float F[4][6][2][3];
```

3.5. Масиви рядків

Варіант 1 — двовимірний масив символів:

```
char S[5][60] = {  
    "Text 1",  
    "Text 2",  
    ...  
};
```

Варіант 2 — масив покажчиків на рядки:

```
char* S[] = {  
    "Str1",  
    "Str2",  
    "Str3"  
};
```

4. Типові задачі з масивами

4.1. Суми та добутки

сума всіх елементів
сума на парних позиціях
сума додатних
добуток непарних

```
sum = 0;  
for (i=0; i<n; i++)  
    sum += A[i];
```

4.2. Пошук максимуму / мінімуму

Пошук максимального значення:

```
max = A[0];  
for (i=1; i<n; i++)  
    if (A[i] > max) max = A[i];
```

Пошук всіх мінімальних елементів:

Збереження індексів у масив.

4.3. Сортування методом вставки

Сортування у спадному порядку:

```
for (i=0; i<9; i++)  
    for (j=i; j>=0; j--)  
        if (A[j] < A[j+1])  
            swap(A[j], A[j+1]);
```

5. Рядки та бібліотечні функції <cstring>

Основні функції:

Функція	Опис
strlen(s)	довжина рядка
strcpy(s1, s2)	копіює s2 → s1
strcat(s1, s2)	додає s2 в кінець s1
strcmp(s1, s2)	порівняння рядків
strchr(s, c)	пошук символу
strstr(s, t)	пошук підрядка

Для використання:

```
#include <cstring>
```

6. Масиви та структури

Масив структур:

```
struct BOOK {  
    char title[50];  
    char author[50];  
    int year;  
    float price;  
};
```

```
BOOK library[5];
```